ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API - интерфейс программирования приложений

DTO - объект передачи данных между слоями приложения

EF Core - средство объектно-реляционного отображения в .NET

JWT - маркер аутентификации и авторизации пользователя

RBAC - разграничение доступа на основе ролей

UI - пользовательский интерфейс

ВВЕДЕНИЕ

Административные функции современного веб-приложения должны обеспечивать не только доступ к данным, но и безопасное управление ролями, контроль пользовательского контента и сопровождение обращений. Для таких задач требуется согласованная работа клиентской и серверной частей, а также корректное хранение данных в базе.

В исходном задании лабораторной работы требуется реализовать управление категориями и ролями пользователей. В настоящем отчете использован близкий по требованиям функциональный аналог на базе проекта NIDSHOP. Проект включает разграничение ролей User, Author и Admin, административную панель, маршруты с ограничением доступа, административный API и сущности, позволяющие классифицировать товарную область по типам и тегам.

Целью работы является анализ и оформление решения, в котором административный модуль реализован по тем же принципам, что и в лабораторном задании: сервер хранит роли и предметные сущности, клиент предоставляет отдельные страницы администратора, а доступ ограничивается на уровне маршрутов и API.

Для достижения цели были поставлены следующие задачи: определить функциональный аналог задания в существующем проекте; выделить серверные сущности, отвечающие за роли и классификацию данных; рассмотреть контроллеры и политики безопасности; описать клиентские страницы администратора; сформировать результат в виде отчета, оформленного в соответствии с требованиями ГОСТ.

1 АНАЛИЗ ЗАДАНИЯ И ВЫБОР ФУНКЦИОНАЛЬНОГО АНАЛОГА

1.1 Постановка задачи

Лабораторная работа 4 ориентирована на развитие административной части веб-приложения. Основные требования сводятся к хранению административно значимых сущностей в базе данных, созданию API для чтения и изменения данных, введению ролевой модели, а также подключению клиентских страниц, доступных только администратору.

В проекте NIDSHOP эти требования реализованы в смежной предметной области. Вместо управления категориями обращений или заявок используются классифицирующие признаки товара и дизайна, а вместо страницы смены ролей присутствует полноценный RBAC-механизм с ролями покупателя, автора и администратора. За счет этого проект можно использовать как корректный практический аналог для описания лабораторной работы.

1.2 Соответствие проекта требованиям лабораторной работы

Таблица 1 - Соответствие исходного задания и выбранного аналога

Требование задания	Реализация в NIDSHOP	Комментарий
Хранение управляемых сущностей в БД	Сущности AppUser, BaseProduct, Design, SupportTicket, SupportMessage	Данные сохраняются через EF Core и PostgreSQL
Роли пользователей и разграничение доступа	UserRole, поле Role в AppUser, политики AdminOnly и AuthorOnly	Доступ проверяется и на сервере, и в клиентском роутинге
Административный API	AdminController и часть методов AuthController	Предусмотрены просмотр тикетов, модерация и сервисные действия
Начальные данные и миграции	Program.cs и папка Migrations	Создаются демо-пользователи с ролями и базовые товары
Клиентские страницы администратора	AdminDashboardPage, AdminModerationPage, AdminSupportPage	Выделен отдельный layout и боковое меню администратора
Интеграция клиента с API	AuthContext и вызовы fetch	В проекте используется не TanStack Query, а прямая интеграция через fetch

Таким образом, проект не повторяет лабораторную работу буквально, однако сохраняет ее ключевую идею: административные функции выделены в отдельный контур, данные хранятся в базе, а доступ к ним регулируется ролями и защищенными маршрутами.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ СЕРВЕРНОЙ ЧАСТИ

2.1 Модель данных и хранение сущностей

В доменной модели ключевой сущностью является AppUser. Пользователь имеет уникальный идентификатор, адрес электронной почты, хеш пароля и роль. Роль хранится в поле Role и использует перечисление UserRole со значениями User, Author и Admin. За счет этого роль фиксируется на уровне данных и может участвовать в генерации JWT-токена и проверке доступа.

Функциональный аналог категории в проекте реализуется через предметные признаки товара и дизайна. Сущность BaseProduct содержит тип изделия и активность, а сущность Design содержит название, статус и набор тегов. Такое решение позволяет классифицировать материалы и подготавливает систему к дальнейшему расширению административных сценариев.

Для работы административного модуля важны также сущности SupportTicket и SupportMessage. Они обеспечивают хранение обращений пользователей и историю взаимодействия с администратором. Все связи и ограничения описываются в ApplicationDbContext, где задаются таблицы, индексы, внешние ключи, каскадные удаления и преобразование перечислений в текстовый вид.

Дополнительной особенностью является использование PostgreSQL и EF Core. Для сущности Design данные о слоях изображения сохраняются в формате JSONB, а для пользователей настроены уникальные индексы по адресу электронной почты и псевдониму автора. Такой подход делает серверную модель устойчивой к дублированию и удобной для последующей модерации.

Место для скриншота схемы данных или структуры backend-модели проекта NIDSHOP

Рисунок 1 - Схема основных сущностей серверной части

2.2 API и механизмы безопасности

Точка входа Program.cs настраивает подключение к PostgreSQL, регистрацию сервисов, JWT-аутентификацию и политики авторизации. Политики AdminOnly, AuthorOnly и UserOnly формируют базовый уровень RBAC и позволяют централизованно ограничивать доступ к методам API.

AuthController обеспечивает регистрацию, вход, обновление профиля, получение данных текущего пользователя и выдачу нового JWT-токена. При авторизации в токен добавляются claims с идентификатором пользователя, адресом электронной почты и текущей ролью. Это позволяет клиенту и серверу опираться на один и тот же источник информации о правах доступа.

AdminController доступен только пользователю с ролью администратора. Через него реализованы методы получения обращений в поддержку, просмотра переписки, ответа на тикет, закрытия обращения, загрузки дизайнов на модерации, а также операций одобрения и отклонения дизайнов. По структуре это соответствует лабораторной работе, где отдельный контроллер предоставляет административные endpoint-ы для управления системой.

Таблица 2 - Основные административные endpoint-ы проекта

Метод и маршрут	Назначение	Доступ
POST /api/auth/login	Авторизация пользователя и получение JWT	Все пользователи

GET /api/auth/me	Получение профиля и роли текущего пользователя	Авторизованный пользователь
GET /api/admin/dashboard	Получение агрегированных метрик панели администратора	Admin
GET /api/admin/moderation/designs	Загрузка списка дизайнов, ожидающих модерации	Admin
POST /api/admin/moderation/designs/{id}/approve	Одобрение публикации дизайна	Admin
POST /api/admin/moderation/designs/{id}/reject	Отклонение дизайна с причиной	Admin
GET /api/admin/support/tickets	Просмотр списка обращений пользователей	Admin
POST /api/admin/support/tickets/{id}/reply	Ответ администратора в выбранный тикет	Admin

Сидирование начальных данных также вынесено на уровень сервера. При первом запуске создаются демо-учетные записи пользователя, автора и администратора, а также базовые товары. Это соответствует требованию лабораторной работы о наличии стартовых ролей и демонстрационных данных.

3 РЕАЛИЗАЦИЯ КЛИЕНТСКОЙ ЧАСТИ

3.1 Маршрутизация и контроль доступа на клиенте

Клиентская часть проекта разработана на React и TypeScript. В файле routes.tsx определены три логических контура: пользовательский, авторский и административный. Каждый контур подключает собственный layout и набор страниц, благодаря чему интерфейс для разных ролей отделен не только визуально, но и структурно.

Компонент RoleGuard выполняет проверку роли перед отображением маршрута. Если неавторизованный пользователь пытается открыть закрытый раздел, он перенаправляется на главную страницу. Если роль не соответствует требованиям маршрута, выполняется редирект в доступный для пользователя раздел. Такой механизм является клиентским аналогом серверной авторизации и повышает удобство работы с интерфейсом.

Контекст AuthContext хранит токен, данные пользователя и текущую роль в localStorage. При инициализации приложения он запрашивает сведения о пользователе через /api/auth/me и восстанавливает сессию. За счет этого административный интерфейс становится доступен только после подтверждения роли, а состояние аутентификации сохраняется между перезагрузками страницы.

Место для скриншота главной административной страницы с метриками и боковым меню

Рисунок 2 - Панель администратора после входа под ролью Admin

3.2 Страницы административного интерфейса

Страница AdminDashboardPage служит точкой входа в административный модуль. На ней показываются агрегированные показатели, в том числе число дизайнов на модерации и число открытых обращений. Дополнительно отображаются укороченные списки объектов, требующих внимания. Это делает панель управления удобным центром принятия решений.

Место для скриншота формы модерации, где администратор может просмотреть, одобрить или отклонить дизайн

Рисунок 3 - Страница модерации дизайнов

Страница AdminModerationPage реализует сценарий обработки пользовательского контента. Администратор видит очередь дизайнов, открывает предпросмотр, а затем принимает решение об одобрении или отклонении работы. При отклонении указывается причина, которая сохраняется в системе. По логике это близко к задаче управления категориями и состояниями сущностей из лабораторной работы.

Место для скриншота списка тикетов и окна переписки администратора с пользователем

Рисунок 4 - Страница административной поддержки

Страница AdminSupportPage содержит список обращений, их статус и переписку по каждому тикету. Администратор может отправить ответ, закрыть обращение и, при необходимости, выполнить административное действие по апелляции на дизайн. Таким образом, интерфейс не ограничивается просмотром данных, а позволяет управлять жизненным циклом пользовательского запроса.

3.3 Сопоставление с исходным заданием

Отдельного внимания заслуживает отличие выбранного аналога от буквальной формулировки лабораторной работы. В проекте NIDSHOP клиентская интеграция построена на стандартных вызовах fetch и React Context, тогда как в задании предлагается применять TanStack Query. При этом архитектурный смысл остается тем же: клиент запрашивает данные через API, хранит состояние авторизации и обновляет интерфейс в зависимости от результата запроса.

Аналогично задача управления категориями заменена на управление административно значимыми сущностями предметной области: типами товаров, тегами дизайнов, объектами модерации и обращениями пользователей. Такое смещение предметной области не нарушает учебную цель, поскольку демонстрирует те же технические подходы к проектированию и разработке.

4 ПРОВЕРКА РАБОТОСПОСОБНОСТИ И РЕЗУЛЬТАТЫ

4.1 Контрольные сценарии

Для оценки корректности реализации административного модуля были сформулированы контрольные сценарии, отражающие ключевые пользовательские действия. Эти сценарии позволяют проверить согласованность ролей, маршрутов, серверных endpoint-ов и клиентского интерфейса.

Таблица 3 - Контрольные сценарии проверки административного модуля

№	Сценарий	Ожидаемый результат
1	Вход под учетной записью администратора	Получен JWT, роль Admin сохранена в клиентском состоянии
2	Переход пользователя без прав в раздел /admin	Маршрут блокируется, выполняется перенаправление
3	Открытие панели администратора	Отображаются метрики, меню и списки объектов для обработки
4	Одобрение дизайна на модерации	Статус дизайна меняется на Published, запись исчезает из очереди
5	Отклонение дизайна с причиной	Статус меняется на Rejected, причина фиксируется в сущности
6	Ответ администратора в тикете поддержки	Сообщение сохраняется и отображается в переписке
7	Закрытие обращения	Статус тикета меняется на Closed и отображается в интерфейсе

По результатам анализа структуры проекта можно сделать вывод, что административный модуль реализован последовательно. Роли закреплены в доменной модели и переносятся в JWT, серверные методы защищены политиками, а клиентский интерфейс использует отдельные маршруты и компоненты для администратора.

Место для скриншота успешного запроса к административному endpoint-у или ответа сервера с данными

Рисунок 5 - Проверка административного API с токеном администратора

4.2 Итоговый результат

Рассмотренный функциональный аналог подтверждает возможность реализации лабораторной работы на основе существующего веб-приложения с иной предметной областью. В проекте присутствуют ключевые компоненты, которых требует задание: хранение сущностей в базе данных, разграничение ролей, административный API, выделенные клиентские страницы и стартовые данные.

С инженерной точки зрения решение можно считать завершенным по следующим признакам: сервер и клиент согласованы по модели ролей, административные маршруты отделены от остальных, данные сохраняются в PostgreSQL, а пользовательский интерфейс предоставляет сценарии работы администратора с минимально необходимым набором операций.

ЗАКЛЮЧЕНИЕ

В ходе подготовки отчета был рассмотрен и оформлен функциональный аналог лабораторной работы 4 на базе проекта NIDSHOP. Несмотря на отличие предметной области от формулировки задания, проект демонстрирует те же принципы построения административного модуля: хранение данных в БД, разграничение прав доступа, выделенный административный API и специализированные клиентские страницы.

Проведенный анализ показал, что роль администратора реализована сквозным образом: от доменных сущностей и JWT-аутентификации до защищенных маршрутов и интерфейсов модерации и поддержки. Это делает выбранный проект подходящим примером для оформления учебного отчета по требованиям лабораторной работы.

Итогом работы стал отчет, оформленный по ГОСТ с учетом заданных ограничений: документ не содержит титульного листа, содержания и списка использованных источников, но включает структурированные разделы, таблицы, подписи к рисункам и подготовленные заглушки под будущие скриншоты.